

AYODIA TESTING TEAM · INTERNAL SHARING

QA AI Skills

Cut effort across every testing phase with AI

13

AI Skills

30–70%

Effort Saved

12

Processes Covered

For Dev · BA · Tester · 15 min · April 2026

The problems we had

We spent more time on repetitive work than actually hunting bugs



Repetitive docs every project

Test Plan / Test Case / Report
rewritten 10x per year



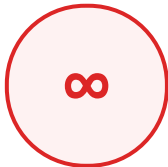
Requirement still unclear

Mid-way through TCs, a gap shows up
→ major revision required



Bug logs incomplete

Dev keeps asking “how to repro?”
→ 3 round-trip loops



Automation can't keep up

100 manual TCs → only 20
automated → coverage slips



Perf analysis is slow

k6 outputs 50 MB JSON →
hunt bottleneck in Excel all day



Weekly updates eat hours

Weekly email to leadership
takes 2 hrs every Friday

The answer — AI drafts, Tester approves



What AI does well

- Drafts structured templates (Plan / TC / Report)
- Surfaces edge cases + boundary / negative tests
- Converts across formats (TC → Playwright code)
- Parses result files (k6 JSON, JMeter CSV)

What humans still own

- Reviewing correctness vs. business intent
- Real Severity / Priority decisions
- Approving before it goes to PM / Dev
- Talking to users during UAT

13 Skills · grouped by testing phase

PRE-SIT

requirement-analyzer

data-type-matrix-generator

SIT — Plan & Design

test-plan-writer

test-matrix-generator

test-case-writer

test-case-reviewer

EXECUTE

bug-report-writer

e2e-test-generator

robot-test-generator

UAT

test-case-writer (uat mode)

test-plan-writer (uat)

test-report-writer (uat)

PERFORMANCE

perf-test-generator

perf-result-analyzer

REPORT & COMM

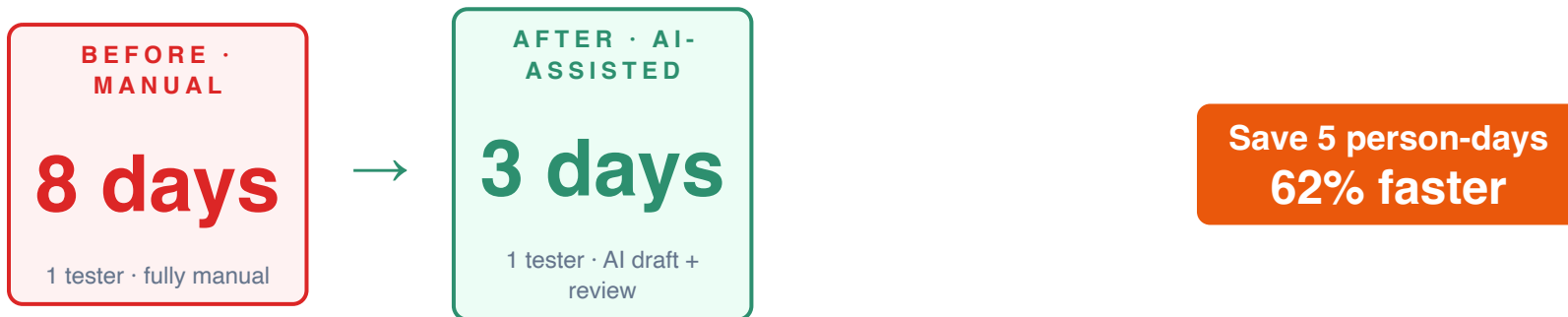
test-report-writer

weekly-update-writer

Every skill shares the same standard (Severity / Priority / Sizing) — data flows across skills with no conversion

Case Study · Login Feature (illustrative)

Hypothetical before/after effort — same module, same scope · numbers illustrate the pattern, not a measured benchmark



Effort breakdown per skill (days)				
Step	Before	After	Cut	What it does
requirement-analyzer	—	0.3	—	Check BRD · 5 Open Qs to PM
test-plan-writer	1.0	0.3	70%	SIT Plan · Exit Criteria · Schedule
test-case-writer	3.0	1.0	67%	28 TCs (positive / negative / boundary)
test-case-reviewer	0.5	0.2	60%	Peer review checklist · 3 fixes
e2e-test-generator	2.0	0.5	75%	Playwright POM · 12 auto scripts
bug-report-writer	0.5	0.2	60%	4 bugs with Severity × Priority
test-report-writer	1.0	0.5	50%	SIT Report · Exit Criteria evaluation

P H A S E 1

Pre-SIT Gate

Check the requirement before testing starts

for BA + Tester Lead

Pre-SIT Gate · check the requirement before testing starts

Prevents the “half a day in, gaps in the requirement force a full redo” situation

requirement-analyzer

Save 30–40%

BA · Tester Lead — gate for BRD / PRD / SRS

Decides whether the requirement is TC-ready:

- Score: Ready / Partial / Not-ready
- Lists “Open Questions” to send back to PM
- Converts to a Normalized Template reused by every skill

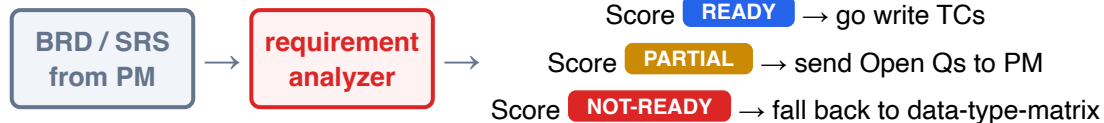
data-type-matrix-generator

Save 30–50%

Tester — fallback when reqs are thin and a deadline looms

Keep moving even when the requirement is incomplete:

- Data Type Matrix (per field × ECP / BVA)
- Happy Path E2E + Integration Points
- Assumption Checklist the PM can tick in 10 min



requirement-analyzer · sample output

Inputs

- BRD / PRD / SRS / user story (raw from PM)
- project-context.md (glossary, business rules)

Outputs (3 files)

1. **Readiness Report** — score + Open Questions
2. **Normalized Requirement** — standard template
3. **PM Confirmation Doc** — checklist for the PM

Why it matters

Cuts the big rework cycle when a requirement gap surfaces late — TCs hit the mark on the first pass

readiness-report.md

Module: LEAVE Management

Source: docs/brd-leave.md

Score: **PARTIAL** (62/100)

Coverage Check

- Functional flow ✓ complete
- Acceptance criteria ⚠ 3 gaps
- Edge cases ⚠ missing (leap year, half-day)
- Security ✗ not addressed

Open Questions (5) Q1. How many half-day slots per day are allowed?

Q2. Does leave overlapping Sat–Sun consume leave days?

Q3. Leap year → +1 day to entitlement?

Q4. Manager hasn't approved in 3 days → escalate?

Q5. Flow for canceling an already-approved leave?

P H A S E 2

SIT

Plan → Design → Review → Execute

for Tester · Tester Lead

SIT · Plan → Design → Review

test-plan-writer

Tester Lead

SIT Plan: Scope · Entry/Exit Criteria · Schedule (Σ Sizing $\times$ 20% buffer) · Risk

Save 40–50%

test-matrix-generator

Tester · before TCs

Coverage / Pairwise / Platform matrix — a quick coverage sanity check when full TCs aren't ready yet

Save —

test-case-writer

Tester · the workhorse

SIT TCs · 23 cols + ECP / BVA / Decision Table / State Transition + Traceability

Save 50–60%

test-case-reviewer

Tester Peer

Automated peer review checklist: clear Expected, Negative/Boundary coverage, no traceability gaps

Save 40%



test-case-writer · sample output

2 Modes

- **SIT mode** — technical, 23 columns
- **UAT mode** — business view or multi-role checklist

Built-in Techniques

- ECP · BVA · Decision Table
- State Transition · Use Case
- Error Guessing

Output formats

Markdown · CSV · TH/EN

How to invoke

Write SIT test cases from
docs/srs-login.md in English CSV
emphasize negative + boundary

testcases_sit_login_20260420.md (4 of 28 cases)

TC ID	Steps (summary)	Expected	Sev	Pri	Auto
AUTH_001	Login with valid email + password	Reaches dashboard	Mi	H	Y
AUTH_002	Login with wrong password 5 times	Lock for 15 min	Cr	Cr	Y
AUTH_003	Invalid email format (no @)	Inline validation error	Mi	M	Y
AUTH_004	SQL injection in email field	Reject + log security event	Cr	Cr	Y

1. Pre-condition · Post-condition · Test Data · Sizing · Traceability · Author · ... (17 more cols)

Chain next: test-case-reviewer (peer review) → e2e-test-generator (auto scripts for TC.Auto=Y)

P H A S E 3

Execute

Bug · Automation

for Tester + Dev

Execute · Bug reports devs read once and get

bug-report-writer

Writes a complete Jira / Linear / GitHub issue in one pass — no follow-up pings:

- Short, specific title
- Environment (browser, build, env)
- Steps to Reproduce (numbered, actually reproducible)
- Expected vs Actual
- **Severity × Priority → Action label**

Example prompt

```
Write a bug report:  
[Checkout] total is wrong when two coupons are stacked  
Chrome 130 on macOS, staging  
Severity: Major
```

Severity × Priority → Action

Severity	Priority	Action
Critical	Critical	Blocker · stop release
Critical	High	Urgent · fix in sprint
Major	High	Standard High
Major	Medium	Plan next sprint
Minor	Low	Backlog
Trivial	Low	Defer / cosmetic

4-level standard from the Ayodia TEST DEFINITION template — identical across every project

e2e-test-generator · sample output

4 Frameworks

- Playwright + TypeScript
- Cypress + TypeScript
- WebDriverIO + TypeScript
- Selenium + Java

Pattern guarantees

- Page Object Model (3-tier)
- Advanced XPath — **no index-based locators**
- Unique vs shared locators split
- Text-as-constants (i18n-ready)

Sister skill

robot-test-generator (Robot Framework, athm_automation pattern)

Generated file structure

```
tests/auth/
├─ login.spec.ts           // test cases
├─ pages/
│  ├─ LoginPage.ts        // page object
│  └─ DashboardPage.ts
├─ locators/
│  └─ login.locators.ts   // XPath constants
└─ constants/
   └─ login.text.ts       // UI text (i18n)
```

login.spec.ts (snippet)

```
test('AUTH_002 · lock account after 5 wrong tries',
  async ({ page }) => {
    const login = new LoginPage(page);
    await login.goto();
    for (let i = 0; i < 5; i++) {
      await login.submit(EMAIL, WRONG_PASSWORD);
    }
    await expect(login.lockedBanner).toBeVisible();
  });
```

P H A S E 4 – 5

UAT + Performance

Business sign-off · Load · Bottleneck

for Tester Lead + Dev

Phase 4–5 · UAT and Performance

UAT

test-case-writer (uat mode)

Save 50–60%

Tester · \$P6

Business view 23 cols **or** UAT Checklist multi-role workflow (officer, finance, manager, ...)

test-report-writer (uat)

Save 60–70%

Tester Lead · \$P8

Total / Pass / Fail / Block / Not Run · Exit Criteria · **User Sign-off section** (name + date + decision)

Approved SIT TCs → auto-converted to UAT TCs (business language) — no starting from scratch

PERFORMANCE

perf-test-generator

Save 50%

Dev / Tester · \$P10

k6 tests (smoke / load / stress / soak / spike) + HTTP wrapper + per-endpoint thresholds

perf-result-analyzer

Save 50%

Tester Lead · \$P11

Parse k6 / JMeter / Gatling → Avg / p95 / p99 + TPS + Error Rate per endpoint + NFR compare + call out **Bottleneck**

Plan(perf) → Generate → Run → Analyze → Report — analysis time drops from half a day to 15 min

A day in the life · Tester using AI Skills

Login feature · 1 sprint day · end-to-end example

- 09:00 ● **/requirement-analyzer** · BRD arrives → run gate → Score Partial · 5 Open Qs sent to PM
- 10:00 ● **(waiting on PM)** · While waiting → **/test-matrix-generator** drafts coverage matrix
- 11:00 ● **/test-plan-writer** · PM replies → SIT Plan + Exit Criteria + Schedule (1.5 days)
- 13:00 ● **/test-case-writer** · Write 28 SIT TCs (positive 12 · negative 10 · boundary 6)
- 14:30 ● **/test-case-reviewer** · Peer review checklist → fix 3 items (unclear Expected)
- 15:00 ● **/e2e-test-generator** · TC.Auto=Y · 12 cases → Playwright POM scripts ready to run
- 16:00 ● **[Execute SIT]** · Run scripts + manual exploratory → find 4 bugs
- 16:30 ● **/bug-report-writer** · Log 4 bugs with full Severity × Priority + Action label in Jira
- 17:00 ● **/test-report-writer** · Draft SIT Report vs. Exit Criteria · send to Tester Lead for review

1 day · 8 skills · 1 feature end-to-end

| AI Guardrails · 5 principles every skill holds to

Keeps AI as an assistant — not the quality judge

1

AI = Draft & Assist · Tester = Review & Approve

AI doesn't judge quality — there's a human gate on every deliverable

2

Always cross-check against the source

TCs traceable back to SRS · Bugs reproduced on a real build · Perf numbers aligned with the NFR

3

Never commit sensitive data

[REDACTED] or env vars — no passwords or PII enter prompts or output

4

Expected Result must be measurable

No “works correctly” / “fast enough” — use numbers or observable behavior

5

Don't make up numbers

No data? Ask the user or mark “TBD” — never guess throughput, response time, or defect counts

Roadmap · what's coming next

NOW · Q2 2026

Shipped (13 skills)

- Pre-SIT gate
- SIT / UAT / Perf full chain
- Bug · Automation · Reporting
- Weekly update

Q3 2026

Mobile + Visual

- mobile-test-generator (Appium)
- visual-regression-checker
- accessibility-auditor (a11y)

Q4 2026

API + Data

- api-contract-tester (Pact)
- test-data-generator (faker + rules)
- db-state-validator

2027

Smart agents

- self-healing locators
- auto-triage agent (Jira)
- flaky-test detector

Want a skill? Open a **PR** against `SKILL-TEMPLATE.md` or post in **#testing-team** — the team will review and add it to the next roadmap cycle

| Get Started

Install (one-time)

1. Install Claude Code:
`npm install -g @anthropic-ai/claude-code`
2. Clone the repo:
`git clone <gitlab>/qa_ai_skill.git`
3. Inside a Claude session:
`/plugin marketplace add /path/to/qa_ai_skill`
`/plugin install qa-ai-skill@ayodia-qa`
4. Verify: `/plugins` → Installed tab

Update

`git pull` → `/plugin marketplace update ayodia-qa`

Further reading

- **New tester?** docs/qa-onboarding.md — 5-min quick start + decision tree
- **Run SIT → UAT?** docs/how-to-sit-uat.md — 10 steps + prompts
- **Input/Process/Output per skill?** docs/work-product-flow.md
- **Standards?** references/qa-standards.md (Sev / Pri / Sizing)
- **Add a new skill?** SKILL-TEMPLATE.md

Contribute

PRs welcome — follow the universal 8-section structure

Questions?

Demo · Discussion · Use cases the team wants to pilot first